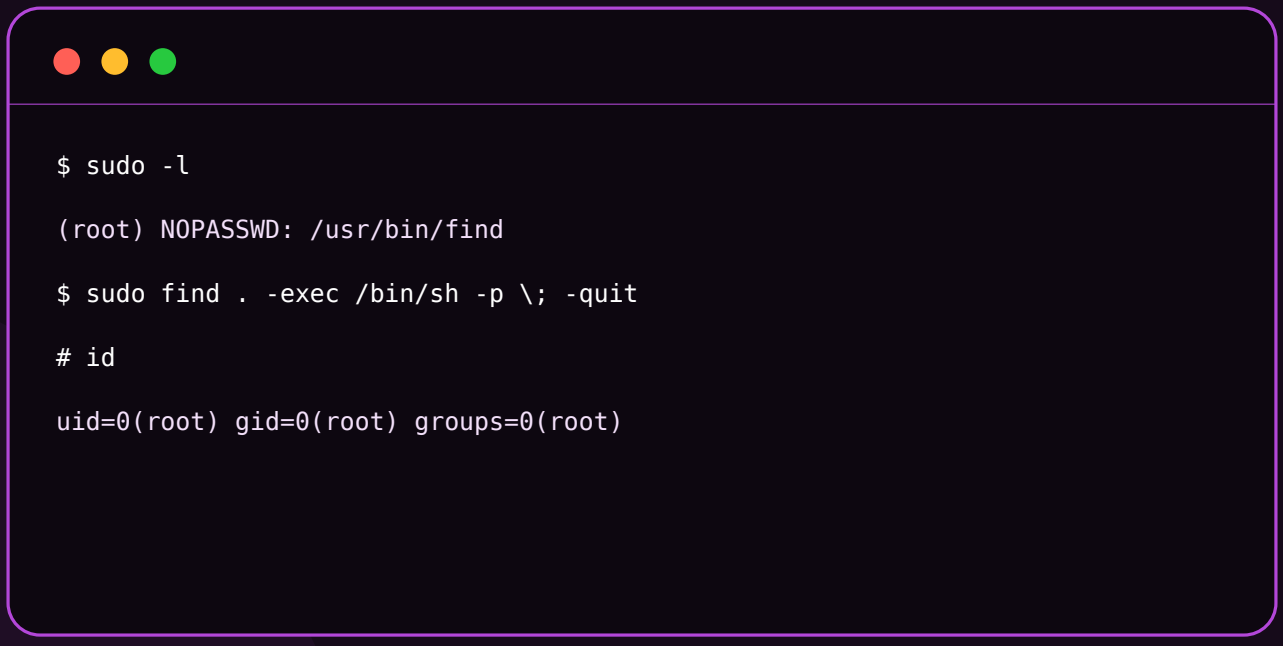
PRAXISHANDBUCH

Linux-PrivEsc

Das Praxishandbuch

Vom Low-Privilege-Zugang zu root – systematische Rechteausweitung unter Linux



SIEBEN KATEGORIEN. EINE FRAGE: WO DARF DAS KONTO MEHR, ALS ES SOLLTE?

Mit Übungen, Spickzettel, Glossar und Index · Ausgabe Oktober 2026

Linux-PrivEsc
Das Praxishandbuch

Vom Low-Privilege-Zugang zu root – systematische Rechteausweitung unter Linux

Von SUID-Binaries über Sudo-Fehlkonfigurationen bis zu Cron-Jobs und Capabilities.
Mit geführten Übungen gegen Metasploitable, Spickzettel, Glossar und Index.

Inhaltsverzeichnis

Vorwort 5

Für wen ist dieses Handbuch? 5

Aufbau 6

Kapitel 1: Was ist Privilege Escalation? 7

1.1 Horizontal und vertikal 7

1.2 Einordnung in den Gesamtablauf 7

1.3 Sieben wiederkehrende Fehlerkategorien 8

Zusammenfassung 8

Kapitel 2: Grundlagen – das Linux-Rechtemodell 9

2.1 Die klassischen Rechte 9

2.2 SUID, SGID und das Sticky Bit 9

2.3 Capabilities: SUID in kleineren Portionen 10

Zusammenfassung 10

Kapitel 3: Installation und Laborumgebung 11

3.1 Laborumgebung 11

3.2 Enumeration-Werkzeuge bereitstellen 11

3.3 Den Ausgangspunkt dokumentieren 12

Zusammenfassung 12

Kapitel 4: Enumeration – manuell und mit Skripten 13

4.1 Manuelle Enumeration: die wichtigsten Fragen 13

4.2 LinPEAS: automatisiert, aber nicht blind vertrauen 13

4.3 Warum manuelle Enumeration trotzdem wichtig bleibt 14

4.4 Alternative Werkzeuge 14

Zusammenfassung 14

Kapitel 5: SUID/SGID-Binaries und GTFOBins 15

5.1 SUID-Binaries finden 15

5.2 GTFOBins: die Referenz für ausnutzbare Standardprogramme 15

5.3 Ein zweites Beispiel: vim 15

5.4 Capabilities statt SUID als ähnlicher Pfad 16

Zusammenfassung 16

Kapitel 6: Sudo-Fehlkonfigurationen . . . 17

6.1 sudo -l als Ausgangspunkt . . . 17

6.2 Typische riskante Muster . . . 17

6.3 Warum NOPASSWD allein kein Fehler ist . . . 17

6.4 Ein Skript als Einfallstor . . . 18

Zusammenfassung . . . 18

Kapitel 7: Cron-Jobs und PATH-Hijacking . . . 19

7.1 Cron-Jobs auffinden . . . 19

7.2 Ein beschreibbares, periodisch laufendes Skript . . . 19

7.3 PATH-Hijacking . . . 19

Zusammenfassung . . . 20

Kapitel 8: Schreibbare Dateien, Dienste und Capabilities . . . 21

8.1 Systemweit beschreibbare Dateien finden . . . 21

8.2 Dienste als Einfallstor . . . 21

8.3 Capabilities praktisch nutzen . . . 22

8.4 Eine Checkliste für dieses Kapitel . . . 22

Zusammenfassung . . . 22

Kapitel 9: Kernel-Schwachstellen – Einordnung und Grenzen dieses Buches . . . 23

9.1 Warum diese Kategorie anders ist . . . 23

9.2 Die Kernel-Version als erster Anhaltspunkt . . . 23

9.3 Warum dieses Buch hier eine Grenze zieht . . . 24

Zusammenfassung . . . 24

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen . . . 25

10.1 Laboraufbau . . . 25

10.2 Checkliste: sichere Laborumgebung . . . 25

10.3 Übungen . . . 25

Zusammenfassung . . . 27

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen . . . 28

11.1 Häufige Probleme . . . 28

11.2 Spickzettel . . . 28

11.3 Glossar . . . 29

11.4 Weiterführende Ressourcen . . . 29

Zusammenfassung . . . 29

Verzeichnis der Befehlsbeispiele **31**

Index **32**

Vorwort

Das Ethical-Hacking- und das Metasploit-Handbuch dieser Reihe erwähnen Rechteausweitung nur knapp: fehlkonfigurierte `sudo`-Regeln, unsichere Dateirechte, ein Post-Modul, das lokale Exploits durchsucht. Dieses Buch macht aus diesen Andeutungen eine vollständige Methodik. Es beantwortet die Frage, die am Ende fast jedes erfolgreichen Web- oder Netzwerkangriffs steht: Du hast einen Zugang mit eingeschränkten Rechten erlangt. Wie wirst du root?

Linux Privilege Escalation ist kein einzelnes Werkzeug, sondern eine systematische Suche nach Fehlkonfigurationen, die ein eingeschränktes Konto in ein uneingeschränktes verwandeln. Dieses Buch ordnet diese Suche in sieben wiederkehrende Kategorien und zeigt zu jeder, woran man sie erkennt und wie man sie im eigenen Labor nachvollzieht.

> **ILLEGAL – niemals ohne Auftrag**
>
> Wie in den anderen Büchern dieser Reihe gilt ohne Ausnahme: Jede Technik in diesem Handbuch ist ausschließlich für den Einsatz in einer eigenen, isolierten Testumgebung oder im Rahmen eines schriftlich autorisierten Auftrags bestimmt. Rechteausweitung setzt bereits einen bestehenden, unautorisierten Zugriff voraus, wenn sie gegen ein fremdes System eingesetzt wird – ein eigenständiger, schwerer Verstoß nach §§ 202a-202c StGB, unabhängig vom späteren Erfolg.

Für wen ist dieses Handbuch?

Du bist hier richtig, wenn du

- bereits das Ethical-Hacking- oder Metasploit-Handbuch kennst und den dort nur knapp behandelten letzten Schritt vertiefen möchtest,
- als angehende Fachkraft für Systemintegration verstehen willst, welche alltäglichen Administrationsfehler ein System tatsächlich verwundbar machen,
- eigene Linux-Server oder -VMs gezielt auf genau diese Fehlerklassen prüfen möchtest,
- dich auf Zertifizierungen vorbereitest, in denen Privilege Escalation einen eigenen Schwerpunkt bildet (eJPT, OSCP).

Aufbau

Kapitel	Thema
1	Was ist Privilege Escalation? Horizontal, vertikal, Einordnung in die Kill Chain
2	Grundlagen: das Linux-Rechtemodell, SUID/SGID, Capabilities
3	Installation und Laborumgebung
4	Enumeration: manuell und mit Skripten
5	SUID/SGID-Binaries und GTFOBins
6	Sudo-Fehlkonfigurationen
7	Cron-Jobs und PATH-Hijacking
8	Schreibbare Dateien, Dienste und Capabilities
9	Kernel-Schwachstellen – Einordnung und Grenzen dieses Buches
10	Praxisteil: Laboraufbau und geführte Übungen
11	Troubleshooting, Spickzettel, Glossar und Ressourcen

> **TIPP – Profi-Trick**
>
> Fast jede Kategorie in diesem Buch lässt sich mit einem einzigen, wiederkehrenden Gedanken zusammenfassen: Irgendwo darf ein Low-Privilege-Konto etwas tun, verändern oder lesen, das eigentlich nur root vorbehalten sein sollte. Die sieben Kapitel unterscheiden sich nur darin, wo genau dieses „Irgendwo" liegt.

Kapitel 1: Was ist Privilege Escalation?

Lernziele

- Du unterscheidest horizontale von vertikaler Rechteausweitung.
- Du kannst Privilege Escalation in den Gesamtablauf eines Penetrationstests einordnen.
- Du kennst die sieben wiederkehrenden Fehlerkategorien, die dieses Buch behandelt.

1.1 Horizontal und vertikal

Rechteausweitung bezeichnet jeden Schritt, bei dem ein bestehender Zugriff auf mehr Rechte oder mehr Daten ausgeweitet wird, als ursprünglich vorgesehen.

Art	Bedeutung	Beispiel
Horizontal	Zugriff auf ein anderes Konto mit vergleichbaren Rechten	vom eigenen Nutzerkonto auf das eines Kollegen mit denselben Berechtigungen
Vertikal	Zugriff auf ein Konto mit höheren Rechten	von einem eingeschränkten Nutzerkonto zu root

Dieses Buch behandelt ausschließlich die **vertikale** Rechteausweitung unter Linux – den Weg von einem bereits erlangten, eingeschränkten Zugang zur vollständigen Kontrolle über das System.

1.2 Einordnung in den Gesamtablauf

Beispiel 1.1: Wo Privilege Escalation im Ablauf steht (vergleiche Ethical-Hacking-Handbuch, Kapitel 3)

```
1. Reconnaissance und Scanning (hping3-, Nmap-Handbuch)
2. Exploitation: ein erster, eingeschränkter Zugang wird erlangt
   (Metasploit-Handbuch, Kapitel 7)
3. Privilege Escalation: eingeschränkter wird zu uneingeschränktem Zugang
   (dieses Buch)
4. Post-Exploitation und Lateral Movement (Metasploit-Handbuch, Kapitel 9)
5. Reporting (Ethical-Hacking-Handbuch, Kapitel 8)
```

Gut zu wissen

Ein Exploit, der einen ersten Zugang verschafft, landet in der Praxis fast nie direkt als root – Webserver-Prozesse, FTP-Dienste und die meisten Netzwerkdienste laufen aus gutem Grund mit eingeschränkten Rechten. Genau diese Lücke zwischen „ein Zugang besteht" und „der Zugang hat volle Rechte" ist der Gegenstand dieses Buches.

1.3 Sieben wiederkehrende Fehlerkategorien

Kategorie	Grundidee	Kapitel
Enumeration	Welche Informationen verrät das System über sich selbst?	4
SUID/SGID-Binaries	Welches Programm läuft mit fremden, höheren Rechten?	5
Sudo-Fehlkonfiguration	Was darf das aktuelle Konto mit `sudo` ausführen, und reicht das für mehr?	6
Cron-Jobs und PATH	Welcher automatisiert laufende Vorgang lässt sich beeinflussen?	7
Schreibbare Dateien/Dienste	Was darf das aktuelle Konto verändern, das root später liest oder ausführt?	7-8
Capabilities	Welches Programm hat gezielt einzelne root-Fähigkeiten, ohne vollständig SUID zu sein?	8
Kernel-Schwachstellen	Ist der laufende Kernel selbst verwundbar?	9

LEGAL – nur mit Auftrag/Einwilligung

Jede dieser sieben Kategorien setzt denselben rechtlichen Rahmen voraus wie der Rest dieser Reihe: eigenes System, autorisierter Auftrag, oder eine dafür vorgesehene Übungsumgebung wie Metasploitable (Kapitel 3).

Zusammenfassung

Zusammenfassung

- Horizontale Rechteausweitung wechselt zwischen gleichrangigen Konten, vertikale führt zu höheren Rechten.
- Dieses Buch behandelt ausschließlich vertikale Rechteausweitung unter Linux, als Schritt nach einem bereits erlangten, eingeschränkten Zugang.
- Sieben wiederkehrende Fehlerkategorien bilden die Struktur der folgenden Kapitel.
- Jede Kategorie lässt sich auf dieselbe Grundfrage zurückführen: Wo darf ein eingeschränktes Konto mehr, als es sollte?

Kapitel 2: Grundlagen – das Linux-Rechtemodell

Lernziele

- Du liest eine `ls -l`-Ausgabe vollständig und korrekt.
- Du verstehst SUID, SGID und das Sticky Bit und ihre jeweilige Wirkung.
- Du kennst den Zweck von Linux Capabilities als feinere Alternative zu SUID.

2.1 Die klassischen Rechte

Beispiel 2.1: Eine typische ls-l-Zeile lesen

```
-rwsr-xr-x 1 root root 157624 Jan 10 09:00 /usr/bin/passwd
```

Abschnitt	Bedeutung
`-` (erstes Zeichen)	Dateityp (`-` regulär, `d` Verzeichnis, `l` Symlink)
`rws`	Rechte des Eigentümers: lesen, schreiben, ausführen – das `s` statt `x` zeigt das SUID-Bit (Kapitel 2.2)
`r-x`	Rechte der Gruppe
`r-x`	Rechte aller übrigen Nutzer
`root root`	Eigentümer-Nutzer und Eigentümer-Gruppe

2.2 SUID, SGID und das Sticky Bit

Bit	Wirkung bei einer ausführbaren Datei
SUID (Set User ID)	das Programm läuft mit den Rechten seines **Eigentümers**, nicht mit denen der ausführenden Person
SGID (Set Group ID)	das Programm läuft mit den Rechten seiner **Eigentümergruppe**; bei einem Verzeichnis erben neu angelegte Dateien dessen Gruppe
Sticky Bit	in einem Verzeichnis dürfen Nutzer nur eigene Dateien löschen, selbst bei Schreibrecht auf das ganze Verzeichnis (z. B. `/tmp`)

Beispiel 2.2: Warum /usr/bin/passwd SUID braucht (Konzept)

```
Ein normaler Nutzer darf /etc/shadow nicht direkt beschreiben.
passwd läuft dank SUID als root und darf es stellvertretend tun,
nachdem es das aktuelle Passwort selbst geprüft hat.
```

Gut zu wissen

SUID ist an sich kein Fehler, sondern ein notwendiger, absichtlich eingebauter Mechanismus – `passwd` selbst ist das Paradebeispiel für einen korrekten, eng begrenzten SUID-Einsatz. Zum Problem wird SUID erst, wenn ein Programm damit mehr kann, als für seinen eigentlichen Zweck nötig wäre (vertieft in Kapitel 5).

2.3 Capabilities: SUID in kleineren Portionen

Capabilities teilen die traditionell an root gebundenen Fähigkeiten in einzelne, separat vergebbare Rechte auf – ein Programm kann zum Beispiel die Fähigkeit erhalten, einen privilegierten Netzwerkport zu öffnen, ohne gleich vollständige root-Rechte zu benötigen.

Beispiel 2.3: Capabilities einer Datei anzeigen

```
$ getcap /usr/bin/ping
/usr/bin/ping cap_net_raw+ep
```

Dieselbe Capability (`cap_net_raw`) ist dir bereits aus dem hping3-Handbuch (Kapitel 3.2) als Alternative zu dauerhaften root-Rechten für Rohsockets bekannt – hier siehst du dieselbe Technik aus der Perspektive der Systemhärtung statt der Werkzeugnutzung.

Zusammenfassung

Zusammenfassung

- Eine `ls -l`-Zeile zeigt Dateityp, Rechte für Eigentümer, Gruppe und Andere sowie den Eigentümer selbst.
- SUID lässt ein Programm mit den Rechten seines Eigentümers laufen, SGID mit denen seiner Gruppe.
- SUID ist an sich kein Fehler, sondern ein notwendiger, eng einzugrenzender Mechanismus.
- Capabilities teilen root-Fähigkeiten feiner auf und sind oft die sicherere Alternative zu vollständigem SUID.

Kapitel 3: Installation und Laborumgebung

Lernziele

- Du richtest einen Low-Privilege-Zugang auf deiner Metasploitable-VM ein.
- Du installierst die Enumeration-Skripte, die in Kapitel 4 verwendet werden.
- Du verifizierst, dass dein Ausgangszugang tatsächlich eingeschränkt ist.

3.1 Laborumgebung

Dieses Buch nutzt dieselbe Metasploitable-2-VM wie die anderen Bände dieser Reihe, im selben isolierten Internal Network (Ethical-Hacking-Handbuch, Kapitel 9). Als Ausgangspunkt dient ein Low-Privilege-Zugang, wie er sich zum Beispiel über die bereits bekannte vsftpd-Backdoor erlangen lässt (Metasploit-Handbuch, Kapitel 7.1).

Beispiel 3.1: Ausgangszugang verifizieren

```
$ id
uid=1001(msfadmin) gid=1001(msfadmin) groups=1001(msfadmin)
$ sudo -l
[sudo] password for msfadmin:
```

Gut zu wissen

Metasploitable 2 ist bewusst so konfiguriert, dass das Standardkonto `msfadmin` bereits viele der in diesem Buch behandelten Schwachstellen aufweist. Für realistischere Übungen kannst du dir zusätzlich eine eigene, bewusst verwundbar konfigurierte Debian- oder Ubuntu-VM aufbauen – die Technik bleibt identisch, nur die konkreten Fehlkonfigurationen unterscheiden sich.

3.2 Enumeration-Werkzeuge bereitstellen

Beispiel 3.2: LinPEAS auf die Kali-VM herunterladen

```
$ wget https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh
```

Beispiel 3.3: Das Skript auf die Ziel-VM übertragen

```
$ python3 -m http.server 8000          # auf der Kali-VM, im Download-Ordner
# Auf der Ziel-VM:
$ wget http://<kali-ip>:8000/linpeas.sh
$ chmod +x linpeas.sh
```

LEGAL – nur mit Auftrag/Einwilligung

Übertrage und starte Enumeration-Skripte ausschließlich auf Systemen, die dir selbst gehören oder für die ein autorisierter Auftrag vorliegt. Ein solches Skript durchsucht das gesamte Dateisystem nach sensiblen Informationen – im eigenen Labor ein Lerninhalt, gegen ein fremdes System ein weiterer, eigenständiger Verstoß.

3.3 Den Ausgangspunkt dokumentieren

Beispiel 3.4: Ausgangssituation vor der Übung festhalten

```
$ whoami
msfadmin
$ hostname
metasploitable
```

> **TIPP – Profi-Trick**
>
> Lege dir vor jeder Übung in Kapitel 10 einen Snapshot der Ziel-VM an. Rechteausweitung verändert oft Dateien oder Konfigurationen dauerhaft – ein Snapshot erlaubt, denselben Ausgangspunkt für die nächste Übung wiederherzustellen.

Zusammenfassung

> **Zusammenfassung**
>
> - Dasselbe Metasploitable-2-Labor wie in den anderen Büchern dieser Reihe dient als Ausgangspunkt.
> - Ein Low-Privilege-Zugang, etwa über die bekannte vsftpd-Backdoor, ist die Voraussetzung für alle folgenden Kapitel.
> - LinPEAS wird über einen einfachen HTTP-Server von der Kali-VM auf die Ziel-VM übertragen.
> - Ein Snapshot vor jeder Übung erlaubt, denselben Ausgangspunkt beliebig oft wiederherzustellen.

Kapitel 4: Enumeration – manuell und mit Skripten

Lernziele

- Du sammelst systematisch Systeminformationen von Hand.
- Du setzt LinPEAS gezielt ein und liest seine Ausgabe.
- Du verstehst, warum manuelle Enumeration trotz guter Skripte wichtig bleibt.

4.1 Manuelle Enumeration: die wichtigsten Fragen

Beispiel 4.1: Grundlegende Systeminformationen

```
$ uname -a
$ cat /etc/os-release
$ id
$ sudo -l
```

Frage	Befehl
Welcher Kernel läuft?	`uname -a`
Welche Distribution und Version?	`cat /etc/os-release`
Welchen Konten und Gruppen gehöre ich an?	`id`
Was darf ich mit sudo?	`sudo -l` (Kapitel 6)
Welche SUID-Programme existieren?	`find / -perm -4000 -type f 2>/dev/null` (Kapitel 5)
Welche Cron-Jobs laufen?	`cat /etc/crontab`, `ls /etc/cron.*` (Kapitel 7)
Was darf ich beschreiben, das root gehört?	`find / -writable -type f 2>/dev/null` (Kapitel 7-8)

4.2 LinPEAS: automatisiert, aber nicht blind vertrauen

Beispiel 4.2: LinPEAS ausführen und Ausgabe sichern

```
$ ./linpeas.sh | tee linpeas_ausgabe.txt
```

LinPEAS durchsucht das System automatisiert nach denselben Kategorien, die auch in diesem Buch behandelt werden, und färbt besonders interessante Funde farblich hervor (rot für hohe Wahrscheinlichkeit einer tatsächlichen Schwachstelle, gelb für prüfenswerte Kandidaten).

WARNUNG – Sicherheitsrisiko

LinPEAS erzeugt oft eine sehr lange, auf den ersten Blick unübersichtliche Ausgabe mit zahlreichen Fehlalarmen. Rot markierte Funde sind ein guter erster Anlaufpunkt, ersetzen aber nicht das Verständnis aus den Kapiteln 5 bis 8 – ohne dieses Verständnis bleibt unklar, warum ein Fund überhaupt relevant ist oder wie er sich ausnutzen lässt.

4.3 Warum manuelle Enumeration trotzdem wichtig bleibt

Gut zu wissen

Ein automatisiertes Skript kann nur nach bekannten Mustern suchen. Eine ungewöhnliche, selbst geschriebene Anwendung mit einer firmenspezifischen Fehlkonfiguration übersieht LinPEAS möglicherweise vollständig. Die manuellen Befehle aus Kapitel 4.1 bleiben deshalb die verlässlichere Grundlage, das Skript liefert vor allem Tempo und Vollständigkeit bei den bekannten, häufigen Fehlerklassen.

4.4 Alternative Werkzeuge

Werkzeug	Charakteristik
LinPEAS	sehr umfangreich, farblich markiert, Teil der PEASS-ng-Sammlung
linux-smart-enumeration (lse)	wählbare Detailtiefe, kompaktere Ausgabe
LinEnum	älter, einfacher aufgebaut, gut zum Nachvollziehen der Grundlogik geeignet

Zusammenfassung

Zusammenfassung

- Manuelle Enumeration beantwortet dieselben sieben Grundfragen, die auch automatisierte Skripte stellen.
- LinPEAS automatisiert diese Suche und hebt wahrscheinlich relevante Funde farblich hervor.
- Automatisierte Skripte erkennen nur bekannte Muster und übersehen firmenspezifische Eigenheiten.
- LinEnum und linux-smart-enumeration sind gängige Alternativen mit jeweils eigenem Schwerpunkt.

Kapitel 5: SUID/SGID-Binaries und GTFOBins

Lernziele

- Du findest SUID-Binaries auf einem System systematisch.
- Du nutzt GTFOBins, um zu prüfen, ob ein gefundenes Programm ausnutzbar ist.
- Du vollziehst ein konkretes, bekanntes Beispiel nach.

5.1 SUID-Binaries finden

Beispiel 5.1: Alle SUID-Programme auflisten

```
$ find / -perm -4000 -type f 2>/dev/null
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/find
```

Die meisten gefundenen Einträge sind unproblematisch und für ihre SUID-Rechte bewusst vorgesehen (`passwd`, `sudo`). Interessant wird es, wenn ein **unerwartetes** Programm in dieser Liste auftaucht – etwa ein für administrative Zwecke mit SUID versehenes `find`, `vim` oder `python3`.

5.2 GTFOBins: die Referenz für ausnutzbare Standardprogramme

GTFOBins ist eine kuratierte, öffentliche Sammlung, die zu zahlreichen gewöhnlichen Unix-Programmen dokumentiert, wie sie sich missbrauchen lassen, wenn sie mit SUID-Rechten, über `sudo` ohne Passwort, oder in bestimmten anderen privilegierten Kontexten erreichbar sind.

Beispiel 5.2: find mit SUID ausnutzen (Konzept, GTFOBins-Eintrag)

```
$ find . -exec /bin/sh -p \; -quit
```

`find` kann im Rahmen seiner `-exec`-Option einen beliebigen Befehl ausführen. Trägt `find` selbst SUID-Rechte, erbt die gestartete Shell (`-p` erhält dabei bewusst die effektive Nutzer-ID) dieselben, erhöhten Rechte.

ILLEGAL – niemals ohne Auftrag

Teste dieses und jedes weitere GTFOBins-Beispiel ausschließlich gegen deine eigene Metasploitable-VM oder eine vergleichbare Laborumgebung, niemals gegen ein System, dessen SUID-Konfiguration du nicht selbst absichtlich dafür eingerichtet hast.

5.3 Ein zweites Beispiel: vim

Beispiel 5.3: vim mit SUID ausnutzen (Konzept, GTFOBins-Eintrag)

```
$ vim -c ':!/bin/sh'
```

Auch hier gilt dasselbe Grundprinzip wie bei `find`: vim darf im Rahmen seiner eigenen Funktionalität beliebige Shell-Befehle ausführen; trägt vim SUID-Rechte, läuft die so gestartete Shell mit denselben, erhöhten Rechten.

5.4 Capabilities statt SUID als ähnlicher Pfad

Beispiel 5.4: Programme mit gesetzten Capabilities finden

```
$ getcap -r / 2>/dev/null
```

GTFOBins dokumentiert für viele Programme zusätzlich, welche Capability (statt vollständigem SUID) zu demselben Ergebnis führt – etwa `cap_setuid` bei `python3`, das denselben Effekt wie ein SUID-Bit erzielt, aber über den in Kapitel 2.3 beschriebenen, feingranularen Mechanismus.

TIPP – Profi-Trick

Prüfe bei jedem auffälligen Fund zuerst auf gtfobins.github.io, ob für genau dieses Programm bereits ein dokumentierter Weg existiert, bevor du selbst nach einem suchst – die Sammlung deckt die meisten gängigen Unix-Programme bereits ab.

Zusammenfassung

Zusammenfassung

- `find / -perm -4000` listet alle SUID-Programme eines Systems auf.
- Die meisten Treffer sind unproblematisch; auffällig sind unerwartete Programme in dieser Liste.
- GTFOBins dokumentiert für zahlreiche Standardprogramme, wie SUID-, sudo- oder Capability-Rechte ausgenutzt werden können.
- Capabilities wie `cap_setuid` führen oft zum selben Ergebnis wie ein SUID-Bit, nur gezielter vergeben.

Kapitel 6: Sudo-Fehlkonfigurationen

Lernziele

- Du liest die Ausgabe von `sudo -l` korrekt und vollständig.
- Du erkennst die häufigsten riskanten sudo-Regelmuster.
- Du verstehst, warum NOPASSWD allein noch kein Fehler ist.

6.1 sudo -l als Ausgangspunkt

Beispiel 6.1: Eigene sudo-Rechte prüfen

```
$ sudo -l
User msfadmin may run the following commands on metasploitable:
    (root) NOPASSWD: /usr/bin/find
```

Diese eine Zeile ist für sich bereits ausreichend: `msfadmin` darf `find` als root ausführen, ohne erneut ein Passwort einzugeben. Mit dem GTFOBins-Wissen aus Kapitel 5.2 lässt sich daraus sofort eine vollständige root-Shell ableiten.

Beispiel 6.2: sudo-Fehlkonfiguration ausnutzen (Konzept)

```
$ sudo find . -exec /bin/sh -p \; -quit
```

6.2 Typische riskante Muster

Muster	Problem
Ein mächtiges, generisches Programm erlaubt (`find`, `vim`, `python3`, `less`)	fast immer ein dokumentierter GTFOBins-Weg zur Shell vorhanden
Ein Platzhalter-Pfad (`/usr/bin/*`) statt eines konkreten Programms	erlaubt faktisch jedes Programm in diesem Verzeichnis
Ein Skript, das vom aufrufenden Nutzer beschrieben werden darf	das Skript selbst lässt sich vor der Ausführung verändern
Eine Umgebungsvariable bleibt bei `sudo` erhalten (`env_keep`)	kann genutzt werden, um das Verhalten des aufgerufenen Programms zu manipulieren

6.3 Warum NOPASSWD allein kein Fehler ist

Gut zu wissen

`NOPASSWD` bedeutet lediglich, dass für diesen einen, konkret erlaubten Befehl keine erneute Passworteingabe nötig ist – das ist in vielen legitimen Automatisierungsszenarien sinnvoll (ein Überwachungsdienst, der gezielt einen einzelnen, harmlosen Befehl als root ausführen muss). Zum Problem wird es erst in Kombination mit einem Befehl, der selbst zu viel kann, wie in Kapitel 6.2 gezeigt.

6.4 Ein Skript als Einfallstor

Beispiel 6.3: Ein per sudo erlaubtes, aber beschreibbares Skript (Konzept)

```
$ sudo -l
(root) NOPASSWD: /opt/scripts/backup.sh
$ ls -l /opt/scripts/backup.sh
-rwxrwxrwx 1 root root ... /opt/scripts/backup.sh
```

Darf der aktuelle Nutzer dieses Skript selbst beschreiben, genügt es, eine eigene Befehlszeile einzufügen, bevor es mit `sudo` erneut ausgeführt wird – das Skript läuft dann mit root-Rechten, unabhängig vom ursprünglich beabsichtigten Inhalt.

LEGAL – nur mit Auftrag/Einwilligung

Jede dieser Techniken gehört ausschließlich in die eigene, dafür vorgesehene Laborumgebung. Das gezielte Verändern eines fremden, per sudo ausführbaren Skripts ist unabhängig von der verwendeten Technik ein eigenständiger, schwerer Eingriff.

Zusammenfassung

Zusammenfassung

- `sudo -l` zeigt direkt, welche Befehle das aktuelle Konto als root ausführen darf.
- Generische, mächtige Programme in dieser Liste lassen sich meist über GTFOBins zur vollständigen Shell ausbauen.
- NOPASSWD allein ist kein Fehler, erst die Kombination mit einem zu mächtigen oder beschreibbaren Befehl.
- Ein per sudo erlaubtes, aber für den aufrufenden Nutzer beschreibbares Skript ist ein eigenständiges Einfallstor.

Kapitel 7: Cron-Jobs und PATH-Hijacking

Lernziele

- Du findest automatisiert laufende Cron-Jobs und ihren Ausführungskontext.
- Du verstehst, wie ein ungeschützter PATH-Aufruf innerhalb eines Skripts ausgenutzt werden kann.
- Du erkennst, warum ein beschreibbares Skript, das root periodisch ausführt, besonders wertvoll ist.

7.1 Cron-Jobs auffinden

Beispiel 7.1: Systemweite Cron-Konfiguration ansehen

```
$ cat /etc/crontab
$ ls /etc/cron.d/ /etc/cron.daily/
```

Beispiel 7.2: Einen Cron-Eintrag lesen

```
*/5 * * * * root /opt/scripts/cleanup.sh
```

Dieser Eintrag lässt `cleanup.sh` alle fünf Minuten **mit root-Rechten** laufen – unabhängig davon, wer das Skript ursprünglich geschrieben hat oder welche Rechte das Skript selbst inzwischen trägt.

7.2 Ein beschreibbares, periodisch laufendes Skript

Beispiel 7.3: Schreibrechte auf das Skript prüfen

```
$ ls -l /opt/scripts/cleanup.sh
-rwxrwxrwx 1 root root ... /opt/scripts/cleanup.sh
```

Darf der aktuelle Nutzer dieses Skript beschreiben, genügt es, eine zusätzliche Zeile einzufügen und bis zu fünf Minuten auf den nächsten automatischen Lauf zu warten – ein vollständig passiver, nicht einmal ausgelöster Weg zu root.

TIPP – Profi-Trick

Ein Cron-basierter Weg zu root ist besonders unauffällig, weil der eigene Zugang zu keinem Zeitpunkt selbst den privilegierten Befehl startet – das System tut es von sich aus, zum nächsten geplanten Zeitpunkt.

7.3 PATH-Hijacking

Ruft ein mit erhöhten Rechten laufendes Skript ein Programm **ohne absoluten Pfad** auf, sucht die Shell dieses Programm entlang der Verzeichnisse in der Umgebungsvariable `PATH` – in genau der Reihenfolge, in der sie dort eingetragen sind.

Beispiel 7.4: Ein verwundbares Skript (Konzept)

```
#!/bin/bash
# /opt/scripts/backup.sh, von root per Cron ausgeführt
tar cf /backup/daten.tar /home
```

Beispiel 7.5: PATH-Hijacking ausnutzen (Konzept)

```
$ echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' > /tmp/tar
$ chmod +x /tmp/tar
$ export PATH=/tmp:$PATH
# Beim nächsten Cron-Lauf von backup.sh wird /tmp/tar statt /usr/bin/tar
# gefunden, sofern /tmp in der für den Cron-Job geltenden PATH-Variable
# vor dem eigentlichen Verzeichnis steht
```

WARNUNG – Sicherheitsrisiko

PATH-Hijacking funktioniert nur, wenn das verwundbare Skript tatsächlich ohne absoluten Pfad aufgerufen wird **und** das manipulierte Verzeichnis in der für diesen Kontext geltenden `PATH`-Variable vor dem eigentlichen Programmverzeichnis steht. Cron-Jobs nutzen oft eine eigene, eingeschränktere `PATH`-Variable als eine interaktive Shell – prüfe das jeweilige `PATH` des konkreten Kontexts, bevor du von einer allgemeinen Shell-Einstellung ausgehst.

Zusammenfassung

Zusammenfassung

- Cron-Jobs mit root-Kontext sind ein wertvolles Ziel, wenn das aufgerufene Skript selbst beschreibbar ist.
- Ein beschreibbares, periodisch laufendes Skript führt zu einem passiven, nicht selbst ausgelösten Weg zu root.
- PATH-Hijacking nutzt aus, dass ein Programm ohne absoluten Pfad entlang der `PATH`-Variable gesucht wird.
- Der tatsächlich geltende `PATH` eines Cron-Kontexts unterscheidet sich oft von dem einer interaktiven Shell.

Kapitel 8: Schreibbare Dateien, Dienste und Capabilities

Lernziele

- Du findest systemweit beschreibbare Dateien, die root später liest oder lädt.
- Du erkennst, wann ein laufender Dienst selbst ein Einfallstor darstellt.
- Du wendest das Capabilities-Wissen aus Kapitel 2.3 praktisch an.

8.1 Systemweit beschreibbare Dateien finden

Beispiel 8.1: Schreibbare Dateien außerhalb des eigenen Home-Verzeichnisses

```
$ find / -writable -type f 2>/dev/null | grep -v "^/home\|^/tmp\|^/proc"
```

Besonders interessant sind Konfigurationsdateien, die von einem mit höheren Rechten laufenden Dienst gelesen werden, sowie Bibliotheken oder Module, die ein privilegierter Prozess nachlädt.

Beispiel 8.2: Eine beschreibbare Datei in /etc/passwd prüfen (Extremfall)

```
$ ls -l /etc/passwd
-rw-rw-rw- 1 root root ... /etc/passwd
```

Gut zu wissen

Ist `/etc/passwd` selbst beschreibbar (ein seltener, aber historisch dokumentierter Fehlkonfigurationsfall), lässt sich ein neuer Eintrag mit UID 0 hinzufügen und damit direkt ein root-äquivalentes Konto anlegen – kein Umweg über ein Programm oder einen Dienst nötig.

8.2 Dienste als Einfallstor

Beispiel 8.3: Mit welchen Rechten laufen aktive Prozesse?

```
$ ps aux | grep root
```

Läuft ein Dienst als root und lädt dabei eine Konfigurationsdatei oder ein Modul, das der aktuelle Nutzer beschreiben kann, überträgt sich dieselbe Logik wie bei einem beschreibbaren Cron-Skript (Kapitel 7.2) auf den Dienst: Die nächste Konfigurationsneuladung oder der nächste Neustart führt den manipulierten Inhalt mit den Rechten des Dienstes aus.

ILLEGAL – niemals ohne Auftrag

Das gezielte Verändern der Konfiguration eines fremden, laufenden Dienstes ist unabhängig vom technischen Weg ein eigenständiger, schwerer Eingriff und gehört ausschließlich in die eigene Laborumgebung.

8.3 Capabilities praktisch nutzen

Beispiel 8.4: Ausnutzbare Capabilities finden

```
$ getcap -r / 2>/dev/null
/usr/bin/python3.8 cap_setuid+ep
```

Beispiel 8.5: cap_setuid ausnutzen (Konzept, GTFOBins-Eintrag)

```
$ python3.8 -c 'import os; os.setuid(0); os.system("/bin/bash")'
```

Trägt `python3` die Capability `cap_setuid`, kann das Programm selbst seine effektive Nutzer-ID auf 0 (root) setzen – unabhängig davon, mit welchen Rechten es ursprünglich aufgerufen wurde.

8.4 Eine Checkliste für dieses Kapitel

Prüfschritt	Befehl
Schreibbare Dateien außerhalb des eigenen Bereichs	`find / -writable -type f 2>/dev/null`
Mit root laufende Prozesse	`ps aux`, gefiltert nach root
Vergebene Capabilities systemweit	`getcap -r / 2>/dev/null`

Zusammenfassung

Zusammenfassung

- Beschreibbare Konfigurationsdateien, die ein privilegierter Prozess später liest, sind ein wiederkehrendes Einfallstor.
- Läuft ein Dienst als root und ist seine Konfiguration beschreibbar, überträgt sich das Risiko auf den nächsten Neustart oder die nächste Neuladung.
- Capabilities wie `cap_setuid` erlauben einem Programm, seine eigene Nutzer-ID direkt auf root zu setzen.
- `find`, `ps` und `getcap` decken gemeinsam die wichtigsten Prüfschritte dieses Kapitels ab.

Kapitel 9: Kernel-Schwachstellen – Einordnung und Grenzen dieses Buches

Lernziele

- Du verstehst, warum Kernel-Exploits eine andere Risikokategorie sind als die vorherigen Kapitel.
- Du kannst die Kernel-Version eines Systems ermitteln und gegen bekannte Schwachstellen einordnen.
- Du weißt, warum dieses Buch bewusst keinen lauffähigen Kernel-Exploit zeigt.

ILLEGAL – niemals ohne Auftrag

Ein Kernel-Exploit greift den Betriebssystemkern selbst an und kann im Fehlerfall das gesamte System zum Absturz bringen, nicht nur den eigenen Zugang gefährden. Dieses Kapitel erklärt das Prinzip anhand eines seit Jahren bekannten, öffentlich dokumentierten und auf modernen Systemen längst behobenen Falls. Es enthält bewusst keinen lauffähigen Exploit-Code. Jede praktische Anwendung gehört ausschließlich in eine Wegwerf-VM, von der du einen Snapshot besitzt.

9.1 Warum diese Kategorie anders ist

Die Kapitel 5 bis 8 nutzen Fehlkonfigurationen auf Anwendungsebene: falsch vergebene Rechte, unsichere Automatisierung, zu großzügige sudo-Regeln. Ein **Kernel-Exploit** setzt dagegen direkt an einer Schwachstelle im Betriebssystemkern selbst an – meist in einem Speicherverwaltungsfehler, der sich gezielt auslösen lässt.

Eigenschaft	Fehlkonfiguration (Kapitel 5-8)	Kernel-Exploit
Ursache	falsch vergebene Rechte, unsichere Automatisierung	ein Programmierfehler im Kernel selbst
Risiko für das Zielsystem	gering bis moderat	ein fehlgeschlagener Versuch kann das System zum Absturz bringen
Reichweite der Behebung	eine einzelne Konfiguration ändern	ein vollständiges Kernel-Update einspielen

9.2 Die Kernel-Version als erster Anhaltspunkt

Beispiel 9.1: Kernel-Version ermitteln

```
$ uname -r
2.6.24-16-server
```

Beispiel 9.2: Gegen bekannte Schwachstellen-Datenbanken abgleichen (Konzept)

```
Kernel-Version in eine Suchmaschine oder eine CVE-Datenbank eingeben,
z. B. "linux kernel 2.6.24 privilege escalation cve"
```

Gut zu wissen

Ein bekanntes, oft als Lehrbeispiel genanntes Beispiel ist **Dirty COW** (CVE-2016-5195), ein 2016 öffentlich bekannt gewordener Fehler in der Speicherverwaltung mehrerer Linux-Kernel-Versionen. Der Fehler wurde zeitnah behoben; moderne, aktuell gepflegte Systeme sind seit Jahren nicht mehr betroffen. Metasploitable 2 nutzt einen bewusst alten Kernel und eignet sich deshalb als historisches Anschauungsbeispiel dafür, wie ein Versionsabgleich zu einem passenden, öffentlich dokumentierten Fund führt.

9.3 Warum dieses Buch hier eine Grenze zieht

Ein lauffähiger Kernel-Exploit unterscheidet sich von den GTFOBins-Beispielen in Kapitel 5 dadurch, dass er nicht nur das eigene Konto betrifft, sondern die Stabilität des **gesamten** Systems aufs Spiel setzt. Dieses Buch zeigt deshalb bewusst nur das Prinzip der Versionserkennung und des Abgleichs, nicht den eigentlichen Exploit-Code.

TIPP – Profi-Trick

Fertige, in Metasploit eingebundene Module für dokumentierte Kernel-Schwachstellen (Metasploit-Handbuch, Kapitel 2.4) sind der verantwortungsvollere Weg, eine solche Schwachstelle im eigenen Labor nachzuvollziehen, als ein von einer unbekannten Quelle heruntergeladenes Einzelskript – sie durchlaufen zumindest eine gewisse Qualitätsprüfung durch das Projekt.

Zusammenfassung

Zusammenfassung

- Kernel-Exploits greifen den Betriebssystemkern direkt an und bergen ein höheres Risiko für die Systemstabilität als Fehlkonfigurationen.
- Die Kernel-Version ist der erste Anhaltspunkt für einen Abgleich mit bekannten, dokumentierten Schwachstellen.
- Dirty COW dient als historisches, längst behobenes Beispiel für diese Fehlerklasse.
- Dieses Buch zeigt bewusst nur das Prinzip, nicht lauffähigen Kernel-Exploit-Code.

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen

Lernziele

- Du baust das für dieses Buch nötige Labor auf.
- Du arbeitest acht Übungen durch, die alle vorherigen Kapitel praktisch verbinden.
- Du weißt, dass derselbe Weg zu root oft über mehrere unterschiedliche Kategorien gleichzeitig möglich ist.

10.1 Laboraufbau

Dieses Buch nutzt dieselbe Metasploitable-2-VM wie die anderen Bände dieser Reihe, erreichbar über einen bereits erlangten Low-Privilege-Zugang (Kapitel 3).

10.2 Checkliste: sichere Laborumgebung

Checkliste 10.1: Vor jeder Übung

```
[ ] Snapshot der Ziel-VM vorhanden
[ ] Internal Network ohne Internetzugang (Ethical-Hacking-Handbuch, Kapitel 9)
[ ] Enumeration-Skripte nur auf die eigene Ziel-VM übertragen
[ ] Jeder gefundene Weg wird dokumentiert, bevor die VM zurückgesetzt wird
```

10.3 Übungen

Jede Übung folgt demselben Schema: Ziel, Voraussetzungen, Durchführung, erwartetes Ergebnis, Auswertung und Fehlerbehebung.

Übung 1: Low-Privilege-Zugang verifizieren

Ziel: Den Ausgangspunkt für alle weiteren Übungen sauber dokumentieren. **Voraussetzungen:** Zugang über die vsftpd-Backdoor (Metasploit-Handbuch, Kapitel 7.1). **Durchführung:** Führe `id`, `whoami` und `sudo -l` aus (Beispiel 3.1). **Erwartetes Ergebnis:** Ein eingeschränktes Konto ohne direkte root-Rechte. **Auswertung:** Notiere Benutzername, UID und die Ausgabe von `sudo -l` als Ausgangszustand. **Fehlerbehebung:** Erhältst du bereits root, hat der Exploit mehr Rechte verschafft als erwartet – für diese Übungsreihe ist ein eingeschränkter Ausgangspunkt nötig, stelle den Snapshot zurück.

Übung 2: Manuelle Enumeration gegen LinPEAS vergleichen

Ziel: Beide Enumeration-Wege nebeneinander erleben. **Voraussetzungen:** LinPEAS auf die Ziel-VM übertragen (Beispiel 3.2-3.3). **Durchführung:** Führe zuerst die manuellen Befehle aus Beispiel 4.1 aus, danach `./linpeas.sh` (Beispiel 4.2). **Erwartetes Ergebnis:** LinPEAS bestätigt und ergänzt die manuell gefundenen Informationen. **Auswertung:** Liste, welche Funde LinPEAS zusätzlich zu deiner manuellen Suche gemeldet hat. **Fehlerbehebung:** Bricht LinPEAS ab, prüfe die Ausführungsrechte der Datei (`chmod +x linpeas.sh`).

Übung 3: Einen SUID-Fund über GTFOBins ausnutzen

Ziel: Von einem gefundenen SUID-Binary zu einer root-Shell gelangen. **Voraussetzungen:** Ergebnis aus Übung 2. **Durchführung:** Suche SUID-Programme (Beispiel 5.1), schlage ein gefundenes, unerwartetes Programm auf gtfobins.github.io nach, und wende den dokumentierten Weg an. **Erwartetes Ergebnis:** Eine Shell mit root-Rechten. **Auswertung:** Prüfe mit `id`, dass die neue Shell tatsächlich als root läuft. **Fehlerbehebung:** Kein unerwartetes SUID-Programm gefunden? Metasploitable 2 enthält mehrere bekannte Kandidaten; vergleiche deine Fundliste mit öffentlich dokumentierten Metasploitable-PrivEsc-Anleitungen.

Übung 4: Eine sudo-Fehlkonfiguration ausnutzen

Ziel: Den Weg aus Kapitel 6 eigenständig nachvollziehen. **Voraussetzungen:** Snapshot zurückgesetzt auf den Zustand nach Übung 1. **Durchführung:** Prüfe `sudo -l` (Beispiel 6.1) und wende den passenden GTFOBins-Weg für den gefundenen, per sudo erlaubten Befehl an. **Erwartetes Ergebnis:** Eine root-Shell, diesmal über sudo statt über ein SUID-Binary. **Auswertung:** Vergleiche den Weg mit Übung 3 – welche Gemeinsamkeit haben beide Wege (GTFOBins)? **Fehlerbehebung:** Der gelistete Befehl hat keinen dokumentierten GTFOBins-Eintrag? Prüfe, ob der Befehl Dateien lesen oder schreiben kann, die sich indirekt für denselben Zweck nutzen lassen.

Übung 5: Einen Cron-Job als Einfallstor nutzen

Ziel: Einen passiven, zeitgesteuerten Weg zu root nachvollziehen. **Voraussetzungen:** Snapshot zurückgesetzt. **Durchführung:** Suche nach beschreibbaren, per Cron ausgeführten Skripten (Beispiel 7.1-7.3) und füge testweise eine harmlose, beobachtbare Zeile ein (z. B. eine Datei anlegen). **Erwartetes Ergebnis:** Die eingefügte Zeile wird beim nächsten Cron-Lauf mit root-Rechten ausgeführt. **Auswertung:** Prüfe Eigentümer und Rechte der neu angelegten Datei – sie sollte root gehören. **Fehlerbehebung:** Nichts passiert nach einigen Minuten? Prüfe das tatsächliche Intervall des Cron-Jobs in `/etc/crontab`.

Übung 6: Eine beschreibbare Konfigurationsdatei finden

Ziel: Kapitel 8.1 praktisch anwenden. **Voraussetzungen:** wie Übung 2. **Durchführung:** Führe `find / -writable -type f 2>/dev/null` aus und filtere offensichtlich irrelevante Pfade heraus (Beispiel 8.1). **Erwartetes Ergebnis:** Mindestens eine Datei außerhalb des eigenen Home-Verzeichnisses, die beschreibbar ist. **Auswertung:** Prüfe, ob ein mit höheren Rechten laufender Prozess diese Datei liest oder ausführt. **Fehlerbehebung:** Lange, unübersichtliche Ausgabe? Filtere zusätzlich nach Verzeichnissen außerhalb von `/tmp` und `/proc`, wie im Beispiel gezeigt.

Übung 7: Capabilities gezielt prüfen

Ziel: Kapitel 8.3 praktisch anwenden. **Voraussetzungen:** wie Übung 2. **Durchführung:** Führe `getcap -r / 2>/dev/null` aus (Beispiel 8.4) und prüfe jeden Treffer gegen GTFOBins. **Erwartetes Ergebnis:** Eine Liste von Programmen mit gesetzten Capabilities, davon mindestens eines potenziell ausnutzbar. **Auswertung:** Vergleiche den Weg über eine Capability mit dem Weg über ein SUID-Binary aus Übung 3 – worin unterscheiden sie sich technisch? **Fehlerbehebung:** Keine interessanten Capabilities gefunden? Das ist ein gültiges, dokumentierbares Ergebnis – nicht jedes System bietet diesen Weg.

Übung 8: Kernel-Version einordnen, ohne auszunutzen

Ziel: Kapitel 9 verantwortungsvoll in die eigene Praxis übertragen. **Voraussetzungen:** keine. **Durchführung:** Führe `uname -r` aus (Beispiel 9.1) und suche die Version zusammen mit „privilege escalation" in einer öffentlichen Schwachstellen-Datenbank. **Erwartetes Ergebnis:** Eine oder mehrere dokumentierte, zur Kernel-Version passende Schwachstellen. **Auswertung:** Notiere, ob für eine gefundene Schwachstelle ein Metasploit-Modul existiert (Metasploit-Handbuch, Kapitel 2), statt einen eigenständigen Exploit-Code zu suchen. **Fehlerbehebung:** Keine passende Schwachstelle gefunden? Das ist bei einem gut gepflegten, aktuellen Kernel das erwartete, positive Ergebnis.

Zusammenfassung

Zusammenfassung

- Dasselbe Metasploitable-2-Labor wie in den anderen Büchern dieser Reihe reicht für alle acht Übungen aus.
- Die Übungen verbinden Enumeration, SUID/GTFOBins, sudo, Cron, beschreibbare Dateien, Capabilities und Kernel-Einordnung.
- Derselbe Zielzustand, root, lässt sich auf Metasploitable über mehrere, unabhängige Wege gleichzeitig erreichen.

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen

Lernziele

- Du kannst die häufigsten Probleme beim Üben von Privilege Escalation einordnen und beheben.
- Du hast einen kompakten Spickzettel für den Alltag zur Hand.
- Du kennst weiterführende Ressourcen für die Vertiefung.

11.1 Häufige Probleme

Symptom	Wahrscheinliche Ursache	Lösung
GTFOBins-Befehl funktioniert nicht wie beschrieben	abweichende Programmversion oder fehlende SUID-Rechte	mit `ls -l` erneut prüfen, ob das SUID-Bit tatsächlich gesetzt ist
`sudo -l` fragt nach einem Passwort, das nicht bekannt ist	das Konto hat keine passwortlosen sudo-Rechte	andere Kategorien (Kapitel 5, 7, 8) statt sudo verfolgen
LinPEAS meldet sehr viele rote Funde gleichzeitig	normales Verhalten auf einem absichtlich unsicheren System wie Metasploitable	Funde priorisieren, mit dem naheliegendsten (z. B. sudo oder SUID) beginnen
Cron-basierter Weg zeigt keine Wirkung	falsches Intervall angenommen oder eigener PATH weicht vom Cron-PATH ab	`/etc/crontab` erneut genau lesen, Cron-eigenen PATH separat prüfen
Erlangte Shell bricht sofort wieder ab	fehlende TTY-Zuweisung bei einer einfachen Shell	`python3 -c 'import pty; pty.spawn("/bin/bash")'` für eine stabilere Shell nutzen

11.2 Spickzettel

Beispiel 11.1: Die wichtigsten Enumeration-Befehle auf einen Blick

```
# Grundinformationen
id
uname -a
sudo -l

# SUID/SGID und Capabilities
find / -perm -4000 -type f 2>/dev/null
getcap -r / 2>/dev/null
# Cron und schreibbare Dateien
cat /etc/crontab
find / -writable -type f 2>/dev/null

# Laufende Prozesse
ps aux
# Automatisierte Enumeration
./linpeas.sh | tee ausgabe.txt
```

11.3 Glossar

- **Capability** – ein einzeln vergebbares, traditionell an root gebundenes Recht, feiner abgestuft als SUID.
- **Cron** – der Linux-Zeitplandienst für automatisiert wiederkehrende Aufgaben.
- **Dirty COW** – eine 2016 bekannt gewordene, längst behobene Linux-Kernel-Schwachstelle (CVE-2016-5195).
- **Enumeration** – die systematische Sammlung von Systeminformationen als Grundlage für die weitere Suche.
- **GTFOBins** – eine öffentliche Sammlung dokumentierter Missbrauchsmöglichkeiten gewöhnlicher Unix-Programme.
- **LinPEAS** – ein weitverbreitetes, automatisiertes Enumeration-Skript aus der PEASS-ng-Sammlung.
- **PATH-Hijacking** – das Ausnutzen eines Programmaufrufs ohne absoluten Pfad über eine manipulierte `PATH`-Variable.
- **Rechteausweitung (Privilege Escalation)** – das Ausweiten eines bestehenden Zugriffs auf mehr Rechte, als ursprünglich vorgesehen.
- **SUID/SGID** – Dateirechte, die ein Programm mit den Rechten seines Eigentümers beziehungsweise seiner Gruppe laufen lassen.

11.4 Weiterführende Ressourcen

Ressource	Inhalt
gtfobins.github.io	die zentrale, kuratierte Referenz für SUID-/sudo-Ausnutzungswege
HackTricks (book.hacktricks.xyz)	sehr umfangreiche, frei verfügbare Sammlung zu Privilege Escalation und vielen weiteren Themen
PayloadsAllTheThings (GitHub)	strukturierte Sammlung von Techniken, einschließlich Linux Privilege Escalation
Ethical-Hacking- und Metasploit-Handbuch (dieselbe Buchreihe)	Gesamtzusammenhang: wie ein Low-Privilege-Zugang überhaupt erst entsteht
TryHackMe / HackTheBox	legale Übungsplattformen mit zahlreichen eigenen Privilege-Escalation-Räumen

TIPP – Profi-Trick

HackTricks und PayloadsAllTheThings ergänzen dieses Buch um deutlich mehr Einzelfälle, als ein einzelner Band abdecken kann – nutze sie als Nachschlagewerk, sobald du auf einen Fund triffst, der hier nicht konkret behandelt wurde.

Zusammenfassung

Zusammenfassung

- Die häufigsten Probleme betreffen abweichende Versionen, fehlende passwortlose sudo-Rechte und unstabile Shells.
- Der Spickzettel bündelt die wichtigsten Enumeration-Befehle aller sieben Kategorien dieses Buches.

- GTFOBins, HackTricks, PayloadsAllTheThings sowie das Ethical-Hacking- und Metasploit-Handbuch sind die wichtigsten Anschlussstellen zum Weiterlernen.

Verzeichnis der Befehlsbeispiele

Beispiel 1.1: Wo Privilege Escalation im Ablauf steht (vergleiche Ethical-Hacking-Handbuch, Kapitel 3) … Seite 7
Beispiel 2.1: Eine typische ls-l-Zeile lesen … Seite 9
Beispiel 2.2: Warum /usr/bin/passwd SUID braucht (Konzept) … Seite 9
Beispiel 2.3: Capabilities einer Datei anzeigen … Seite 10
Beispiel 3.1: Ausgangszugang verifizieren … Seite 11
Beispiel 3.2: LinPEAS auf die Kali-VM herunterladen … Seite 11
Beispiel 3.3: Das Skript auf die Ziel-VM übertragen … Seite 11
Beispiel 3.4: Ausgangssituation vor der Übung festhalten … Seite 12
Beispiel 4.1: Grundlegende Systeminformationen … Seite 13
Beispiel 4.2: LinPEAS ausführen und Ausgabe sichern … Seite 13
Beispiel 5.1: Alle SUID-Programme auflisten … Seite 15
Beispiel 5.2: find mit SUID ausnutzen (Konzept, GTFOBins-Eintrag) … Seite 15
Beispiel 5.3: vim mit SUID ausnutzen (Konzept, GTFOBins-Eintrag) … Seite 15
Beispiel 5.4: Programme mit gesetzten Capabilities finden … Seite 16
Beispiel 6.1: Eigene sudo-Rechte prüfen … Seite 17
Beispiel 6.2: sudo-Fehlkonfiguration ausnutzen (Konzept) … Seite 17
Beispiel 6.3: Ein per sudo erlaubtes, aber beschreibbares Skript (Konzept) … Seite 18
Beispiel 7.1: Systemweite Cron-Konfiguration ansehen … Seite 19
Beispiel 7.2: Einen Cron-Eintrag lesen … Seite 19
Beispiel 7.3: Schreibrechte auf das Skript prüfen … Seite 19
Beispiel 7.4: Ein verwundbares Skript (Konzept) … Seite 20
Beispiel 7.5: PATH-Hijacking ausnutzen (Konzept) … Seite 20
Beispiel 8.1: Schreibbare Dateien außerhalb des eigenen Home-Verzeichnisses … Seite 21
Beispiel 8.2: Eine beschreibbare Datei in /etc/passwd prüfen (Extremfall) … Seite 21
Beispiel 8.3: Mit welchen Rechten laufen aktive Prozesse? … Seite 21
Beispiel 8.4: Ausnutzbare Capabilities finden … Seite 22
Beispiel 8.5: cap_setuid ausnutzen (Konzept, GTFOBins-Eintrag) … Seite 22
Beispiel 9.1: Kernel-Version ermitteln … Seite 23
Beispiel 9.2: Gegen bekannte Schwachstellen-Datenbanken abgleichen (Konzept) … Seite 23
Checkliste 10.1: Vor jeder Übung … Seite 25
Beispiel 11.1: Die wichtigsten Enumeration-Befehle auf einen Blick … Seite 28

Index

cap_net_raw – 10
cap_setuid – 16, 22
Capabilities – 10, 16, 22, 26
Cron – 19, 21, 26, 29
Dirty COW – 29
Enumeration – 11, 13, 14, 25, 28, 29
GTFOBins – 15, 16, 17, 22, 24, 26, 29
Kernel-Exploit – 23, 24
LinPEAS – 11, 13, 25, 29
NOPASSWD – 17
PATH-Hijacking – 19, 20, 29
Privilege Escalation – 5, 7, 27, 29
SGID – 9, 29
Sticky Bit – 9
sudo – 5, 15, 17, 18, 23, 25, 26
SUID – 9, 10, 15, 16, 26, 29

LINUX-PRIVESC DAS PRAXISHANDBUCH

Der letzte Schritt nach dem ersten Zugang

Ein Exploit verschafft selten direkt root. Dieses Handbuch vertieft, was im Ethical-Hacking- und Metasploit-Handbuch nur kurz erwähnt wird: den systematischen Weg von einem eingeschränkten zu einem uneingeschränkten Zugang. Es ordnet die Suche in sieben wiederkehrende Kategorien, von SUID-Binaries über Sudo-Fehlkonfigurationen bis zu Cron-Jobs, und zieht bei Kernel-Exploits bewusst eine klare Grenze.

DAS ERWARTET DICH

- Das Linux-Rechtemodell: SUID, SGID, Sticky Bit und Capabilities verständlich erklärt
- Enumeration von Hand und mit LinPEAS, mit klarer Arbeitsteilung zwischen beidem
- SUID-Binaries und GTFOBins, Sudo-Fehlkonfigurationen systematisch erkannt
- Cron-Jobs, PATH-Hijacking, schreibbare Dateien und Dienste als Einfallstor
- Kernel-Schwachstellen eingeordnet, bewusst ohne lauffähigen Exploit-Code

Für wen?

Leser:innen des Ethical-Hacking- und Metasploit-Handbuchs, die den Schritt von einem eingeschränkten zu einem uneingeschränkten Zugang vertiefen möchten, sowie Studierende und Auszubildende. Ausschließlich für legale, autorisierte Tests und eigene Laborumgebungen.

32 Seiten · Stand Oktober 2026 · Ergänzung zum Ethical-Hacking- und Metasploit-Handbuch

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.